

```
1 #GAN
```

```
1 import tensorflow as tf
2 from tensorflow.keras import layers
3 import matplotlib.pyplot as plt
4 import numpy as np
```

```
1 (x_train,_),(_,_) = tf.keras.datasets.mnist.load_data()
2 x_train = (x_train - 127.5) / 127.5
3 x_train = np.expand_dims(x_train, axis=-1)
```

```
1 def build_generator():
2     model = tf.keras.Sequential()
3     model.add(layers.Dense(7*7*256, use_bias=False, input_shape=(100,)))
4     model.add(layers.BatchNormalization())
5     model.add(layers.LeakyReLU())
6     model.add(layers.Reshape((7, 7, 256)))
7
8     model.add(layers.Conv2DTranspose(128, (5, 5), strides=(1, 1), padding='same', use_bias=False))
9     model.add(layers.BatchNormalization())
10    model.add(layers.LeakyReLU())
11    # Upsample to 28x28 with 64 filters
12    model.add(layers.Conv2DTranspose(64, (5, 5), strides=(2, 2), padding='same', use_bias=False))
13    model.add(layers.BatchNormalization())
14    model.add(layers.LeakyReLU())
15    # Output 28x28 with 1 channel (for grayscale MNIST)
16    model.add(layers.Conv2DTranspose(1, (5, 5), strides=(2, 2), padding='same', use_bias=False, activation='tanh'))
17    return model
```

```
1 def build_discriminator():
2     model = tf.keras.Sequential()
3     model.add(layers.Conv2D(64, (5, 5), strides=(2, 2), padding='same', input_shape=[28, 28, 1]))
4     model.add(layers.LeakyReLU())
5     model.add(layers.Dropout(0.3))
6
7     model.add(layers.Conv2D(128, (5, 5), strides=(2, 2), padding='same'))
8     model.add(layers.LeakyReLU())
9     model.add(layers.Dropout(0.3))
10
11    model.add(layers.Flatten())
12    model.add(layers.Dense(1))
13    return model
```

```
1 #loss function
2 cross_entropy = tf.keras.losses.BinaryCrossentropy(from_logits=True)
3 def generator_loss(fake_output):
4     return cross_entropy(tf.ones_like(fake_output), fake_output)
5 def discriminator_loss(real_output, fake_output):
6     real_loss = cross_entropy(tf.ones_like(real_output), real_output)
7     fake_loss = cross_entropy(tf.zeros_like(fake_output), fake_output)
8     total_loss = real_loss + fake_loss
9     return total_loss
10
11    generator = build_generator()
12    discriminator = build_discriminator()
13
14    gen_optimizer = tf.keras.optimizers.Adam(1e-4)
15    disc_optimizer = tf.keras.optimizers.Adam(1e-4)
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape` to `input
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
1 @tf.function
2 def train_step(images):
3     noise = tf.random.normal([batch_size, 100])
4     with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:
5         generated_images = generator(noise, training=True)
6
7         real_output = discriminator(images, training=True)
8         fake_output = discriminator(generated_images, training=True)
9
```

```
10     gen_loss=generator_loss(fake_output)
11     disc_loss=discriminator_loss(real_output,fake_output)
12
13     gradients_of_generator=gen_tape.gradient(gen_loss,generator.trainable_variables)
14     gradients_of_discriminator=disc_tape.gradient(disc_loss,discriminator.trainable_variables)
15
16     gen_optimizer.apply_gradients(zip(gradients_of_generator,generator.trainable_variables))
17     disc_optimizer.apply_gradients(zip(gradients_of_discriminator,discriminator.trainable_variables))
```

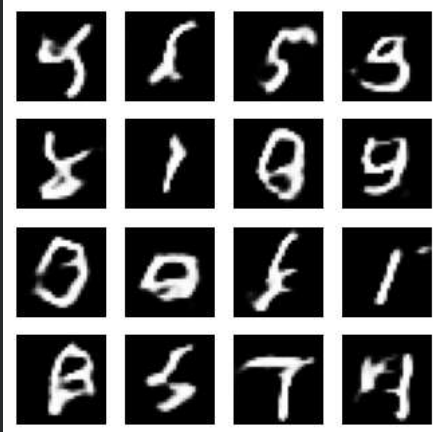
```
1 from re import VERBOSE
2 EPOCHS=50
3 batch_size=256
4 dataset=tf.data.Dataset.from_tensor_slices(x_train).shuffle(60000).batch(batch_size)
5 def train(dataset,epochs):
6     for epoch in range(epochs):
7         for image_batch in dataset:
8             train_step(image_batch)
9
10     print(f'Epoch {epoch+1} completed')
11 train(dataset,EPOCHS)
```

```
Epoch 1 completed
Epoch 2 completed
Epoch 3 completed
Epoch 4 completed
Epoch 5 completed
Epoch 6 completed
Epoch 7 completed
Epoch 8 completed
Epoch 9 completed
Epoch 10 completed
Epoch 11 completed
Epoch 12 completed
Epoch 13 completed
Epoch 14 completed
Epoch 15 completed
Epoch 16 completed
Epoch 17 completed
Epoch 18 completed
Epoch 19 completed
Epoch 20 completed
Epoch 21 completed
Epoch 22 completed
Epoch 23 completed
Epoch 24 completed
Epoch 25 completed
Epoch 26 completed
Epoch 27 completed
Epoch 28 completed
Epoch 29 completed
Epoch 30 completed
Epoch 31 completed
Epoch 32 completed
Epoch 33 completed
Epoch 34 completed
Epoch 35 completed
Epoch 36 completed
Epoch 37 completed
Epoch 38 completed
Epoch 39 completed
Epoch 40 completed
Epoch 41 completed
Epoch 42 completed
Epoch 43 completed
Epoch 44 completed
Epoch 45 completed
Epoch 46 completed
Epoch 47 completed
Epoch 48 completed
Epoch 49 completed
Epoch 50 completed
```

```

1 #generate images
2 def generate_images(model,test_input):
3     predictions=model(test_input,training=False)
4     fig=plt.figure(figsize=(4,4))
5     for i in range(predictions.shape[0]):
6         plt.subplot(4,4,i+1)
7         plt.imshow(predictions[i,:,:,:]*127.5+127.5,cmap='gray')
8         plt.axis('off')
9
10    plt.show()
11 noise=tf.random.normal([16,100])
12 generate_images(generator,noise)

```



```

1 #generate synthetic samples
2 def generate_synthetic_data(generator,num_samples=1000):
3     noise=tf.random.normal([num_samples,100])
4     synthetic_data=generator(noise,training=False)
5     return synthetic_data
6 synthetic_images=generate_synthetic_data(generator,500)
7 print("synthetic dataset shape:",synthetic_images.shape)
8 print(synthetic_images[0:1, 0:1, 0:1, 0:1])

```

```

synthetic dataset shape: (500, 28, 28, 1)
tf.Tensor([[[[-0.9832435]]]], shape=(1, 1, 1, 1), dtype=float32)

```

```

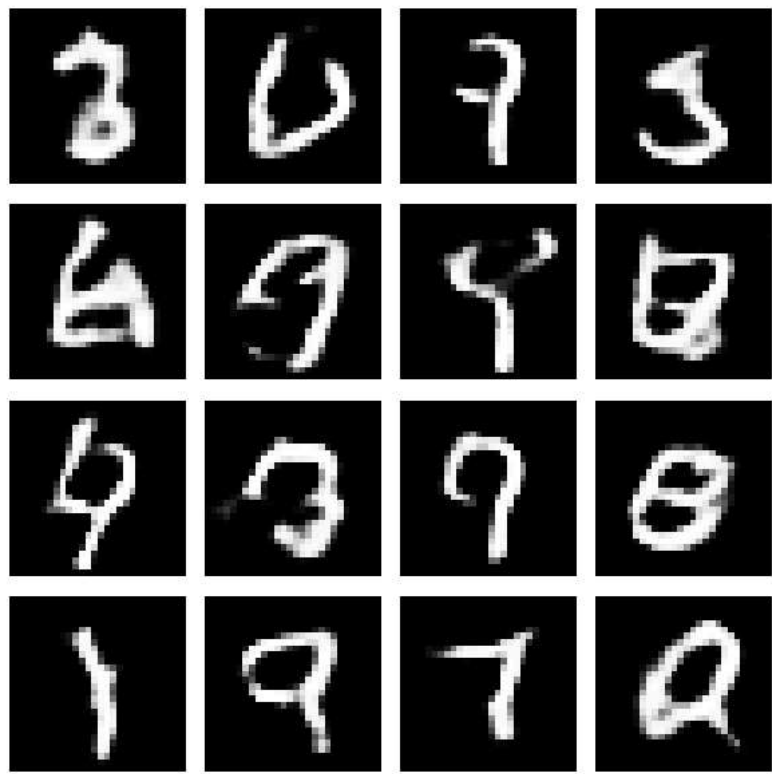
1 synthetic_images=(synthetic_images*127.5+127.5).numpy()
2 combined_data=np.concatenate([x_train[:500],synthetic_images[:500]])

```

```

1 import tensorflow as tf
2 import matplotlib.pyplot as plt
3 def display_synthetic_images(generator,num_images=16):
4     noise=tf.random.normal([num_images,100])
5     generate_images=generator(noise,training=False)
6     generate_images=(generate_images*127.5+127.5)
7
8     plt.figure(figsize=(6,6))
9     for i in range(num_images):
10        plt.subplot(4,4,i+1)
11        plt.imshow(generate_images[i,:,:,:],cmap='gray')
12        plt.axis('off')
13    plt.tight_layout()
14    plt.show()
15 #call function
16 display_synthetic_images(generator,16)

```



1 Start coding or [generate](#) with AI.

```
1 #cifar10
```

```
1 import tensorflow as tf
2 from tensorflow.keras import layers
3 import matplotlib.pyplot as plt
4 import numpy as np
```

```
1 (x_train,_),(_,_) = tf.keras.datasets.cifar10.load_data()
2 x_train = (x_train - 127.5) / 127.5
3 x_train = np.expand_dims(x_train, axis=-1)
```

```
1 def build_generator():
2     model = tf.keras.Sequential()
3     model.add(layers.Dense(7*7*256, use_bias=False, input_shape=(100,)))
4     model.add(layers.BatchNormalization())
5     model.add(layers.LeakyReLU())
6     model.add(layers.Reshape((7, 7, 256)))
7
8     model.add(layers.Conv2DTranspose(128, (5, 5), strides=(1, 1), padding='same', use_bias=False))
9     model.add(layers.BatchNormalization())
10    model.add(layers.LeakyReLU())
11    # Upsample to 28x28 with 64 filters
12    model.add(layers.Conv2DTranspose(64, (5, 5), strides=(2, 2), padding='same', use_bias=False))
13    model.add(layers.BatchNormalization())
14    model.add(layers.LeakyReLU())
15    # Output 28x28 with 1 channel (for grayscale MNIST)
16    model.add(layers.Conv2DTranspose(1, (5, 5), strides=(2, 2), padding='same', use_bias=False, activation='tanh'))
17    return model
```

```

1 def build_discriminator():
2     model=tf.keras.Sequential()
3     model.add(layers.Conv2D(64,(5,5),strides=(2,2),padding='same',input_shape=[28,28,1]))
4     model.add(layers.LeakyReLU())
5     model.add(layers.Dropout(0.3))
6
7     model.add(layers.Conv2D(128,(5,5),strides=(2,2),padding='same'))
8     model.add(layers.LeakyReLU())
9     model.add(layers.Dropout(0.3))
10
11     model.add(layers.Flatten())
12     model.add(layers.Dense(1))
13     return model

```

```

1 #loss function
2 cross_entropy=tf.keras.losses.BinaryCrossentropy(from_logits=True)
3 def generator_loss(fake_output):
4     return cross_entropy(tf.ones_like(fake_output),fake_output)
5 def discriminator_loss(real_output,fake_output):
6     real_loss=cross_entropy(tf.ones_like(real_output),real_output)
7     fake_loss=cross_entropy(tf.zeros_like(fake_output),fake_output)
8     total_loss=real_loss+fake_loss
9     return total_loss
10
11 generator =build_generator()
12 discriminator=build_discriminator()
13
14 gen_optimizer=tf.keras.optimizers.Adam(1e-4)
15 disc_optimizer=tf.keras.optimizers.Adam(1e-4)

```

```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/'input
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

```

```

1 @tf.function
2 def train_step(images):
3     noise=tf.random.normal([batch_size,100])
4     with tf.GradientTape() as gen_tape,tf.GradientTape() as disc_tape:
5         generated_images=generator(noise,training=True)
6
7         real_output=discriminator(images,training=True)
8         fake_output=discriminator(generated_images,training=True)
9
10        gen_loss=generator_loss(fake_output)
11        disc_loss=discriminator_loss(real_output,fake_output)
12
13        gradients_of_generator=gen_tape.gradient(gen_loss,generator.trainable_variables)
14        gradients_of_discriminator=disc_tape.gradient(disc_loss,discriminator.trainable_variables)
15
16        gen_optimizer.apply_gradients(zip(gradients_of_generator,generator.trainable_variables))
17        disc_optimizer.apply_gradients(zip(gradients_of_discriminator,discriminator.trainable_variables))

```

```

1 from re import VERBOSE
2 EPOCHS=50
3 batch_size=256
4 dataset=tf.data.Dataset.from_tensor_slices(x_train).shuffle(60000).batch(batch_size)
5 def train(dataset,epochs):
6     for epoch in range(epochs):
7         for image_batch in dataset:
8             train_step(image_batch)
9
10        print(f'Epoch {epoch+1} completed')
11 train(dataset,EPOCHS)

```

```

Epoch 1 completed
Epoch 2 completed
Epoch 3 completed
Epoch 4 completed
Epoch 5 completed
Epoch 6 completed
Epoch 7 completed
Epoch 8 completed
Epoch 9 completed
Epoch 10 completed
Epoch 11 completed
Epoch 12 completed
Epoch 13 completed

```

```
Epoch 14 completed
Epoch 15 completed
Epoch 16 completed
Epoch 17 completed
Epoch 18 completed
Epoch 19 completed
Epoch 20 completed
Epoch 21 completed
Epoch 22 completed
Epoch 23 completed
Epoch 24 completed
Epoch 25 completed
Epoch 26 completed
Epoch 27 completed
Epoch 28 completed
Epoch 29 completed
Epoch 30 completed
Epoch 31 completed
Epoch 32 completed
Epoch 33 completed
Epoch 34 completed
```

```
1  #generate images
2  def generate_images(model,test_input):
3      predictions=model(test_input,training=False)
4      fig=plt.figure(figsize=(4,4))
5      for i in range(predictions.shape[0]):
6          plt.subplot(4,4,i+1)
7          plt.imshow(predictions[i,:,:,:]*127.5+127.5,cmap='gray')
8          plt.axis('off')
9
10     plt.show()
11     noise=tf.random.normal([16,100])
12     generate_images(generator,noise)
```

```
1  #generate synthetic samples
2  def generate_synthetic_data(generator,num_samples=1000):
3      noise=tf.random.normal([num_samples,100])
4      synthetic_data=generator(noise,training=False)
5      return synthetic_data
6  synthetic_images=generate_synthetic_data(generator,500)
7  print("synthetic dataset shape:",synthetic_images.shape)
8  print(synthetic_images[0:1, 0:1, 0:1, 0:1])
```

```
1 synthetic_images=(synthetic_images*127.5+127.5).numpy()
2 combined_data=np.concatenate([x_train[:500],synthetic_images[:500]])
```